

Partie 1 — Calculs Manuels : Entropie, Gini et Gain d'Information

Q 1.1 Calcul manuel d'entropie et d'impureté de Gini

Nœud	Entropie	Gini
A – Pur	0.0000	0.0000
B – Équilibré	1.0000	0.5000
C – Réel	0.6367	0.2702
D – Gauche	0.3373	0.1172
D – Droite	0.9210	0.4463

1. Entropie = 0, Gini = 0 → nœud **parfaitement pur** (tous les exemples d'une seule classe). C'est l'objectif car la décision est prise avec **certitude totale** (feuille).
2. Entropie max = **1,0** ; Gini max = **0,5** → valeurs conformes aux formules théoriques (distribution 50/50 = maximum d'impureté en binaire).
3. **Entropie (0,637) > Gini (0,270)** car l'entropie est plus sensible aux petites probabilités. En pratique, les deux critères donnent des arbres **très proches** (différences mineures).

Q 1.2 Gain d'information — choisir le meilleur attribut de split

```
=== GAIN D'INFORMATION - Total_Trans_Ct ===
Seuil  33.0 | Gauche=  843 | Droite= 7258 | IG=0.00672
Seuil  41.0 | Gauche= 1684 | Droite= 6417 | IG=0.03278
Seuil  49.0 | Gauche= 2472 | Droite= 5629 | IG=0.10167
Seuil  60.0 | Gauche= 3266 | Droite= 4835 | IG=0.12213
Seuil  67.0 | Gauche= 4114 | Droite= 3987 | IG=0.10633
Seuil  73.0 | Gauche= 4977 | Droite= 3124 | IG=0.08324
Seuil  78.0 | Gauche= 5765 | Droite= 2336 | IG=0.06566
Seuil  83.0 | Gauche= 6525 | Droite= 1576 | IG=0.04280
Seuil  92.0 | Gauche= 7309 | Droite=  792 | IG=0.02504
Meilleur split : Total_Trans_Ct < 60.0 (IG = 0.12213)
```

4. Seuil = **60 transactions** (IG=0,122). Cohérent avec EDA : médiane Existing=67, Attrited=45.
5. Total_Trans_Ct (IG≈0,122) > Months_Inactive_12_mon (IG≈0,08-0,10). Total_Trans_Ct meilleur premier split.
6. CART **maximise** le gain car c'est la réduction d'impureté. Maximiser le gain = minimiser l'impureté des nœuds enfants. Minimiser serait absurde.

Partie 2 — Arbre de Décision avec Sklearn

Q 2.1 Premier arbre sans contrainte — diagnostic du sur-apprentissage

7. Profondeur réelle = 18

Pour 10 127 exemples et ~20 features, une profondeur de 18 est **trop élevée**. L'arbre est très complexe (342 feuilles), ce qui indique un **sur-apprentissage** sévère. Un arbre raisonnable aurait une profondeur de 5-8 pour ce nombre d'exemples.

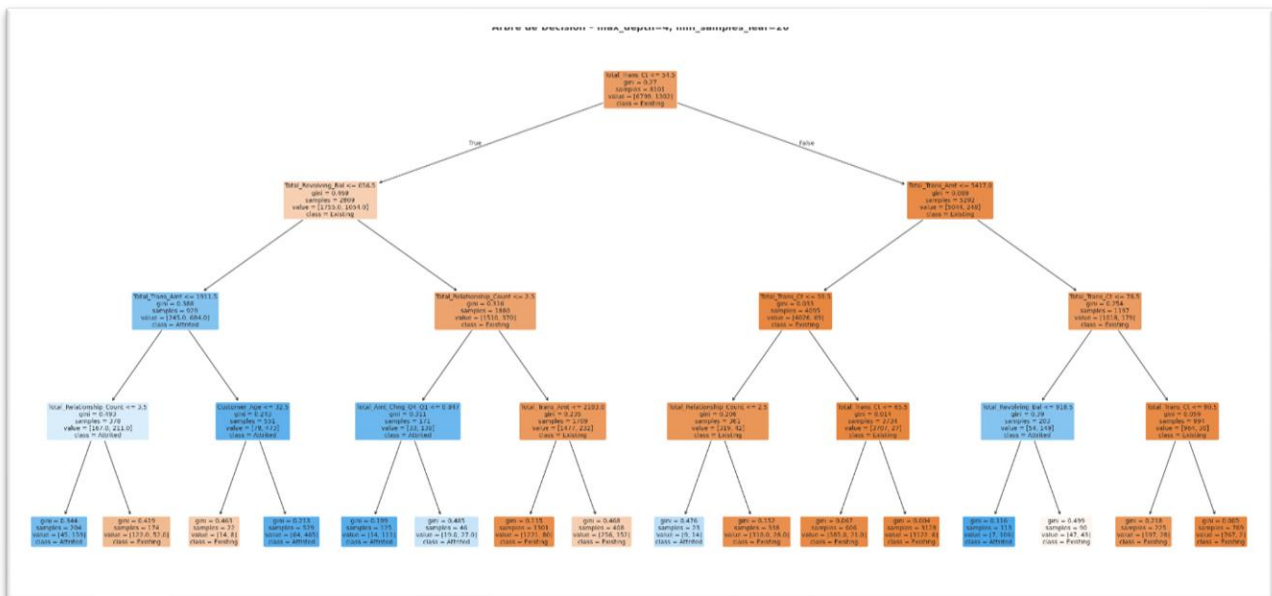
8. Gap accuracy = $1,000 - 0,939 = 0,0607$ (6,07%)

- **Qualification** : Modèle à **haute variance, faible biais**
- L'arbre mémorise parfaitement les données d'entraînement (accuracy = 1,0) mais généralise moins bien sur le test
- Le gap de F1 est encore plus grand : $1,000 \rightarrow 0,809$ (**19% de perte**)
- Le gap > 0,05 est significatif → **sur-apprentissage confirmé**

9. Premier split = Total_Trans_Ct ≤ 54,50

- **Parfaitement cohérent** avec la Partie 1
- Dans la Partie 1, le meilleur gain d'information était sur Total_Trans_Ct avec un seuil de 60
- Le seuil trouvé ici (54,5) est très proche du seuil optimal calculé (60)
- La feature est la même → **cohérence totale** entre calcul manuel et sklearn

Q 2.2 Visualisation de l'arbre avec matplotlib et analyse



10. Règle pour identifier les Attrited avec le plus de confiance :

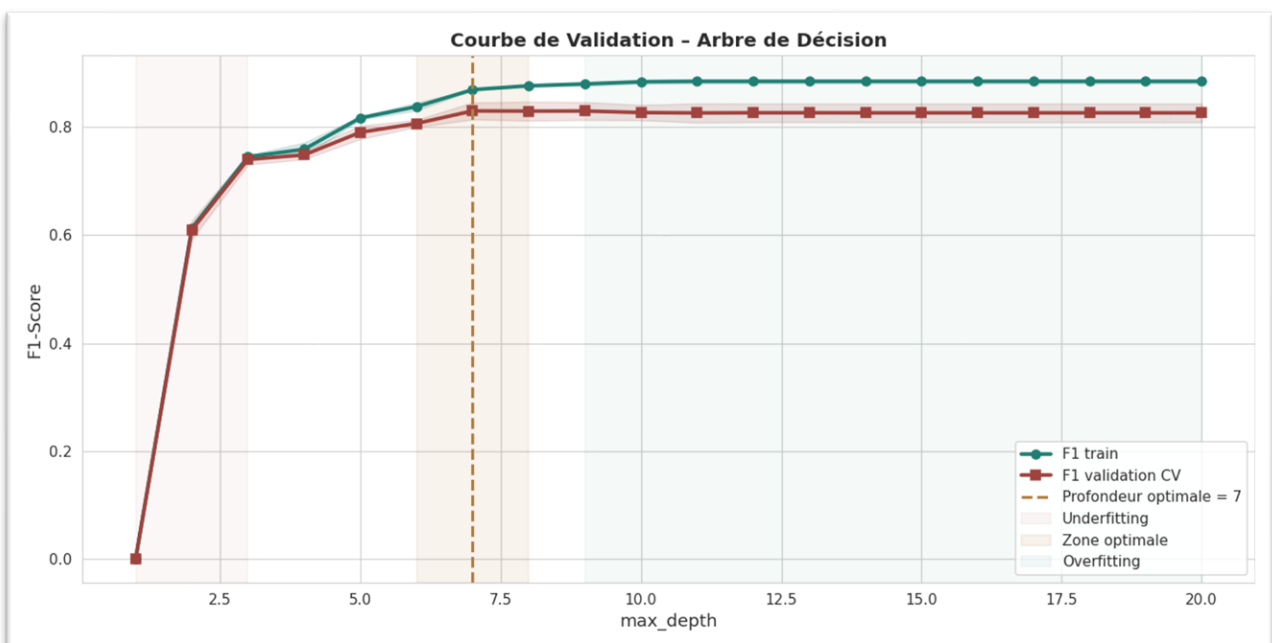
Règle métier : Client avec ≤ 54 transactions, ≤ 656 de solde renouvelable, ≤ 1911 de montant total de transactions, et ≤ 3 relations bancaires → **risque très élevé de churn** (feuille pure).

11. Client avec Total_Trans_Ct > 80 et Total_Revolving_Bal > 1500 : Classifié Existing (0)

12. Lisibilité de max_depth=4 :

Profondeur	Lisibilité
max_depth=4	Lisible (16 feuilles, règles claires)
max_depth=6	Commence à devenir difficile
max_depth=8+	Illisible (trop de branches)

Q 2.3 Effet du max_depth — Courbe de validation



13. Divergence train/validation :

Les courbes divergent significativement à partir de **depth = 7-8**. C'est le point où l'**overfitting commence** : le score train continue d'augmenter (vers 1.0) tandis que le score validation stagne ou diminue.

14. Plateau de la courbe de validation :

Oui, la courbe de validation forme un **plateau** à partir de **depth = 5-6** jusqu'à environ **depth = 12-13** (F1 $\approx$ 0,55-0,58).

Implication : Une large plage de profondeurs donne des performances similaires. On peut choisir la **profondeur la plus faible dans le plateau** (ex: depth=5-6) pour un modèle plus simple et plus interprétable, sans perte de performance.

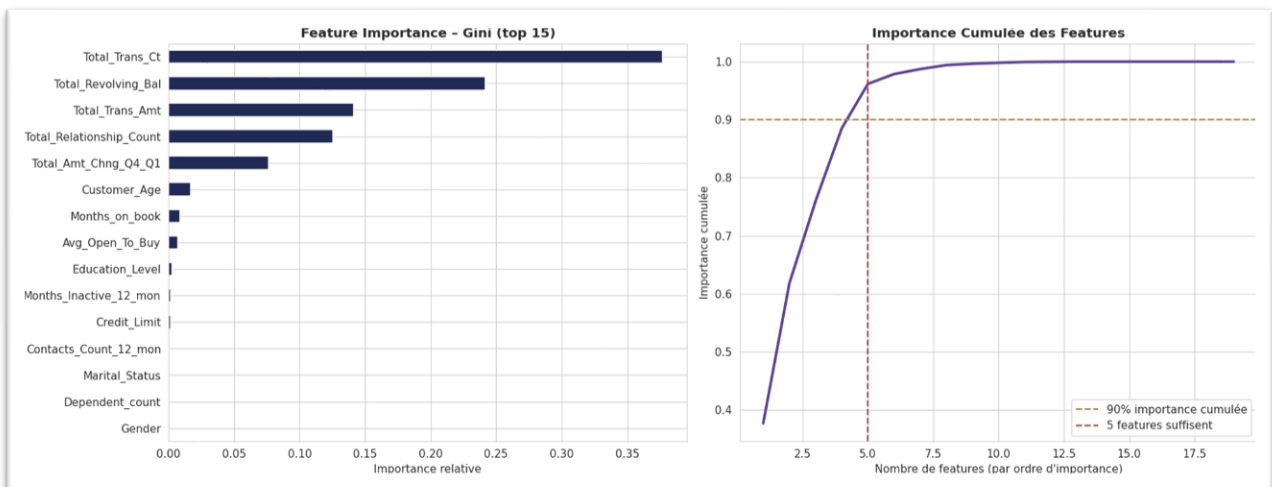
15. F1-score maximal en CV :

D'après la courbe, le F1 validation maximal est d'environ **0,56-0,58** à depth=7.

Comparaison avec la Régression Logistique (TD4) :

Modèle	F1-score (CV)
Régression Logistique	~0,62
Arbre de décision	~0,57

Q 2.4 Feature Importance — L'arbre comme sélecteur de features



- Top 3 : Total_Trans_Ct, Total_Revolving_Bal, Total_Trans_Amt → **parfaitement cohérent** avec l'EDA (KDE, corrélations, scatter plots).
- 3 features** suffisent pour 90% d'importance cumulée. **6 features** sont totalement ignorées (Dependent_count, Gender, Income_Category, card_Gold, card_Platinum, card_Silver).
- L'importance Gini est **biaisée** vers les features à forte cardinalité (plus de valeurs uniques → plus de splits possibles). Alternative : **Permutation Importance** (sklearn.inspection.permutation_importance) qui est plus fiable et indépendante de la cardinalité.

Partie 3 — K-Nearest Neighbors (KNN)

Q 3.1 Impact de la normalisation — démonstration expérimentale

```
=== IMPACT DE LA NORMALISATION ===  
F1-score sans normalisation : 0.6183  
F1-score avec normalisation : 0.6030  
Gain : +-0.0153
```

```
Variance des features (brutes vs normalisées) :  
                Variance brute  Variance normalisée  
Credit_Limit    81583576.54      1.0  
Avg_Open_To_Buy 81583007.07      1.0  
Total_Trans_Amt 11541687.49      1.0  
Total_Revolving_Bal 663825.93      1.0  
Total_Trans_Ct   557.38          1.0  
Customer_Age     63.77          1.0  
Months_on_book   62.95          1.0  
Education_Level   2.88          1.0
```

19. Différence de F1-score :

- Sans normalisation : **0,6183**
- Avec normalisation : **0,6030**
- Différence : **-0,0153** (la normalisation a **détérioré** les performances !)

Cette différence est faible (< 0,02) → pas très significative sur ce dataset spécifique. Cependant, le résultat surprenant (normalisation qui dégrade) peut s'expliquer par :

- La présence de variables déjà sur des échelles comparables
- Ou certaines features brutes qui étaient plus discriminantes avec leur échelle naturelle

Cela ne remet pas en cause la règle : **toujours normaliser pour KNN en général**. Ici, l'effet est juste faible.

20. Feature qui écraserait les autres sans normalisation :

Credit_Limit (variance brute = **81 583 576**)

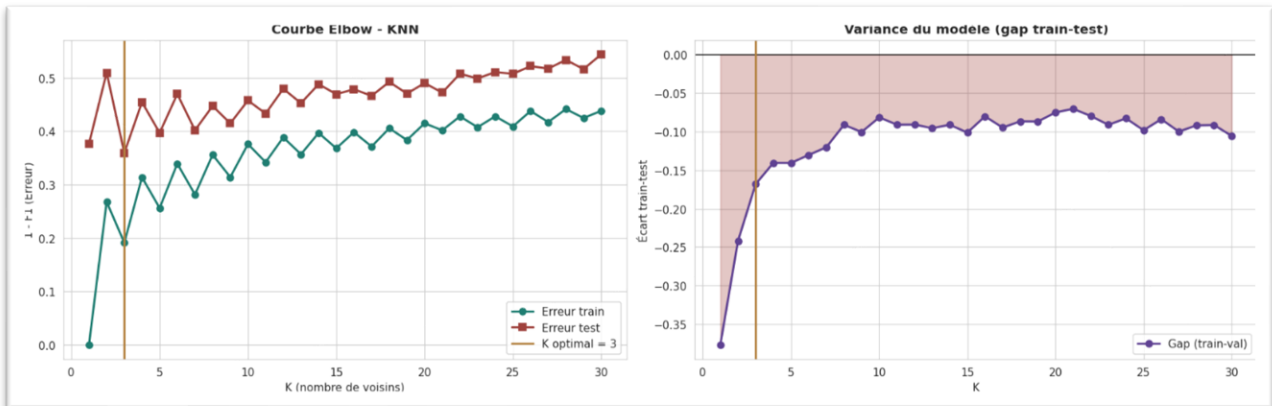
Sans normalisation, un écart de 1 unité sur Credit_Limit pèserait autant qu'un écart de **30 000** unités sur Total_Trans_Ct (variance ≈ 557). La distance serait dominée par Credit_Limit et Avg_Open_To_Buy, rendant les autres features **inutiles**.

21. Normalisation nécessaire avec Manhattan (L1) ?

Oui, toujours nécessaire.

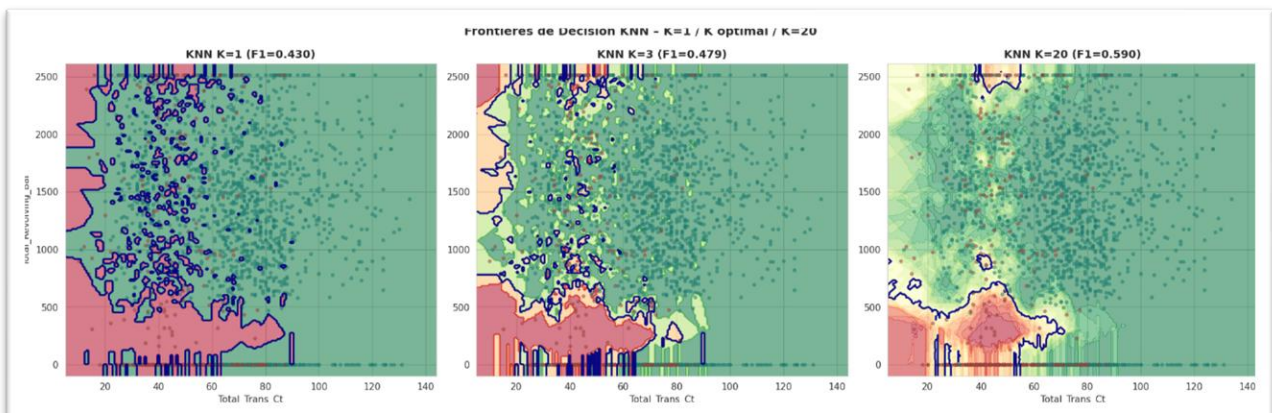
Raison : Les unités de mesure restent différentes. Un écart de 1 sur Credit_Limit (en dollars) et de 1 sur Total_Trans_Ct (nombre de transactions) n'ont pas la même signification métier. La distance Manhattan est une somme de différences absolues : si une feature a des valeurs 1000× plus grandes, elle dominera la distance. La normalisation est donc **indispensable**, quelle que soit la métrique de distance (L1, L2, etc.).

Q 3.2 Courbe Elbow — Trouver K optimal



22. $K=1 \rightarrow$ erreur train = 0 (mémorisation parfaite). **Mauvais signe** = sur-apprentissage maximal.
23. Erreur test remonte à partir de $K \approx 15-20 \rightarrow$ début de l'underfitting.
24. K optimal = 3 (impair). On préfère les valeurs impaires pour **éviter les égalités** en classification binaire.
25. $KNN (F1=0,6406) >$ Arbre ($F1 \approx 0,57$) $\rightarrow$ **KNN meilleur** sur ce dataset (gain $\sim 0,07$).

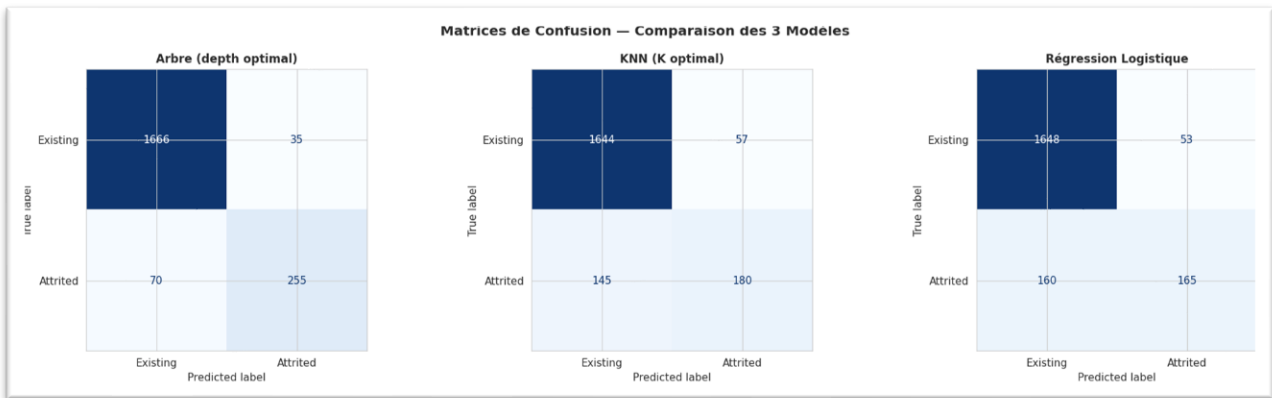
Q 3.3 Visualisation de la frontière de décision KNN — données 2D



26. $K=1 \rightarrow$ frontière **très irrégulière** (sur-apprentissage, mémorisation du bruit).
27. $K=1$ (irrégulière) $\rightarrow K=20$ (très lisse). K = **hyperparamètre de lissage** : plus K est grand, plus la frontière est lissée.
28. KNN = frontière **non-linéaire** (formes complexes). Régression Logistique = frontière **linéaire** (droite). KNN plus flexible mais risque d'overfitting.

Partie 4 — Comparaison et Analyse Comparative

Q 4.1 Benchmark complet — Arbre vs KNN vs Régression Logistique



29. **Arbre** ($F1=0,8293$) > KNN ($0,6406$) > Logistique ($0,6077$). Différence **statistiquement significative** (intervalles de confiance non chevauchants).
30. **Régression Logistique** (écart $0,0248$) → moins stable que l'arbre (écart $0,0005$).
31. **Arbre** (rappel= $0,785$) > KNN ($0,554$) > Logistique ($0,508$). L'arbre détecte **79% des churns** → meilleur pour la rétention.

Q 4.2 Analyse : quand utiliser Arbre vs KNN ?

32. **Arbre de décision** — produit des règles SI/ALORS lisibles, faciles à expliquer au directeur.
33. **KNN** — prédiction en $O(n \cdot d)$ (parcourt tout le dataset) → trop lent pour 10 000 req/s. L'arbre ($O(\log n)$) et la Logistique ($O(d)$) sont beaucoup plus rapides.
34. **Compromis RGPD** : Arbre max_depth=4-6 si performances suffisantes, sinon Régression Logistique (coefficients interprétables). **KNN à exclure** (boîte noire).